

PDF 4 of 4 — Interview Notes: English + Hindi Bilingual

Availability

Every section of the Interview & Practical Notes (PDF 3) — mental models, the nines table, trade-offs, misconceptions, interview questions and answers, follow-ups, exercises, and the senior-engineer perspective — translated paragraph by paragraph into Hindi, with a glossary.

पूरा PDF 3 · Bilingual

Q&A + Follow-ups सहित

Advanced Glossary

Contents / विषय-सूची

1. Topic Overview
2. Simple Mental Model
3. Key Points to Remember
4. Real-World Examples
5. When to Use / When NOT to Use
6. Trade-offs
7. Common Misconceptions
8. Interview Questions
9. Interview Answers
10. Follow-up Questions
11. Practical Design Exercises
12. Quick Interview Revision Sheet
13. Senior Engineer Perspective
14. General + Technical Words Glossary

1. Topic Overview

ENGLISH

Availability is a measure of how accessible and reliable a system stays for its users, expressed as uptime: the share of time the system is actually usable against the time it is promised to be usable. It exists because every real system fails sometimes — hardware dies, networks partition, deployments go wrong, traffic spikes beyond capacity — and availability is the discipline of designing so that individual failures don't become user-visible outages.

हिंदी

Availability यह मापने का एक तरीका है कि कोई सिस्टम अपने users के लिए कितना accessible और reliable बना रहता है, जिसे uptime के रूप में व्यक्त किया जाता है: सिस्टम जितने समय usable होने का वादा करता है, उसमें से वह वाकई कितने समय usable रहता है। यह इसलिए ज़रूरी है क्योंकि हर असली सिस्टम कभी-न-कभी fail होता है — hardware खराब होता है, networks partition हो जाते हैं, deployments गलत हो जाते हैं, traffic capacity से आगे बढ़ जाता है — और availability वह discipline है जिससे design किया जाता है ताकि individual failures, users को दिखने वाले outages न बन जाएँ।

ENGLISH

The problem availability solves is trust: users and downstream systems need to know a service will respond when called, within a promised window, most of the time. A system can be functionally correct and fast, and still be a failure in production if it is frequently unreachable.

हिंदी

Availability जिस समस्या को हल करती है वह है भरोसा: users और downstream systems को यह जानना होता है कि जब भी उन्हें बुलाया जाएगा, एक वादा किए गए समय के भीतर, ज़्यादातर बार एक service जवाब देगी। कोई सिस्टम functionally सही और तेज़ हो सकता है, और फिर भी production में असफल हो सकता है अगर वह बार-बार unreachable रहे।

ENGLISH

Availability matters in System Design because it is usually the metric a business actually pays for — SLAs, customer trust, and revenue are tied to uptime more directly than to almost any other quality attribute. It also sits in tension with other properties covered elsewhere in this course: chasing availability means adding copies of things (redundancy, replication, geographic distribution), and the moment there is more than one copy of data, you inherit a consistency problem.

हिंदी

System Design में Availability इसलिए मायने रखती है क्योंकि यह आमतौर पर वह metric है जिसके लिए business वाकई पैसे देता है — SLAs, customer trust, और revenue, लगभग किसी भी दूसरे quality attribute की तुलना में uptime से कहीं ज़्यादा सीधे जुड़े होते हैं। Availability के पीछे भागने का मतलब है चीज़ों की copies जोड़ना (redundancy, replication, geographic distribution), और जैसे ही data की एक से ज़्यादा copy बन जाती है, आपको एक consistency की समस्या विरासत में मिल जाती है।

ENGLISH

In a larger architecture, availability decisions shape almost everything downstream: how many regions you run in, whether your data store is single-primary or multi-primary, how aggressive your health checks and failover are, and how much engineering and operational cost you're willing to spend chasing the next nine.

हिंदी

एक बड़े architecture में, availability से जुड़े फैसले लगभग हर आगे वाली चीज़ को आकार देते हैं: आप कितने regions में चलते हैं, आपका data store single-primary है या multi-primary, आपके health checks और failover कितने aggressive हैं, और अगली nine पाने के लिए आप कितना engineering और operational खर्च करने को तैयार हैं।

Beginner framing

Availability = the system doesn't go down.

शुरुआती समझ: Availability = सिस्टम down नहीं होता।

Senior framing

Availability = a measured, promised uptime for a specific, named part of the system — achieved by deliberately creating copies of things, which then forces an explicit consistency trade-off.

Senior समझ: Availability = सिस्टम के किसी खास, नामित हिस्से के लिए मापा गया, वादा किया गया uptime — जिसे जान-बूझकर copies बनाकर हासिल किया जाता है, जो एक स्पष्ट consistency trade-off को मजबूर करता है।

2. Simple Mental Model

The analogy: a chain of convenience stores instead of one supermarket

हिंदी – शीर्षक

उपमा: एक बड़े supermarket की जगह convenience stores की एक chain

ENGLISH

Imagine a city served by a chain of small convenience stores instead of one giant supermarket. If one store loses power or sells out of milk, customers just walk to another branch down the street — that's **redundancy**. The chain deliberately builds more stores than the bare minimum, and spreads them across different neighborhoods so one local blackout doesn't take out the whole city — that's **geographic distribution**. A sign (or an app) points customers to whichever nearby store is open and not too crowded — that's **load balancing**. Store managers check stock and equipment on a schedule, and head office gets an alert if a fridge stops working — that's **health monitoring and maintenance**.

हिंदी

एक ऐसे शहर की कल्पना कीजिए जहाँ एक विशाल supermarket की जगह छोटे convenience stores की एक chain सेवा देती है। अगर एक स्टोर की बिजली चली जाए या दूध खत्म हो जाए, तो customers बस सड़क पर आगे किसी दूसरी branch पर चले जाते हैं — यही **redundancy** है। Chain जान-बूझकर ज़रूरत से ज़्यादा स्टोर बनाती है, और उन्हें अलग-अलग इलाकों में फैलाती है ताकि एक स्थानीय बिजली कटौती पूरे शहर को प्रभावित न करे — यही **geographic distribution** है। एक साइन (या एक app) customers को बताता है कि पास का कौन सा स्टोर खुला है — यही **load balancing** है। Store managers नियमित रूप से stock और equipment जाँचते हैं, और अगर कोई फ्रिज काम करना बंद कर दे तो head office को alert मिलता है — यही **health monitoring और maintenance** है।

ENGLISH

The interesting question is the inventory app. When Store A sells the last carton of milk, does the app instantly show "0 left" everywhere (it has to phone every other store's till and wait for confirmation before updating — strong consistency, and the app is briefly slower or unavailable while it waits), or does it update Store A's count immediately and let the citywide total catch up a few seconds later (eventual consistency, so the app stays fast and available, but for a moment a customer elsewhere might be told "3 left" when there are really 2)?

हिंदी

दिलचस्प सवाल inventory app का है। जब Store A का दूध का आखिरी कार्टन बिक जाता है, तो क्या app तुरंत हर जगह "0 बचा है" दिखाता है (इसके लिए हर दूसरे स्टोर के till को फोन करना होगा और confirmation का इंतज़ार करना होगा — strong consistency, और app इंतज़ार में थोड़ी देर के लिए धीमा या unavailable हो जाता है), या यह तुरंत Store A का count अपडेट कर देता है और शहर भर का total कुछ सेकंड बाद catch up करने देता है (eventual consistency, जिससे app तेज़ और available बना रहता है, लेकिन एक पल के लिए कहीं और किसी customer को बताया जा सकता है "3 बचे हैं" जबकि असल में 2 ही हैं)?

Beginner understanding

More servers/locations = more available. Add copies and you're safer.

अधिक servers/locations = ज़्यादा available। Copies जोड़ें और आप ज़्यादा सुरक्षित हैं।

Interview-level understanding

Copies only help if they fail independently. And every copy you add is a new place data can disagree — so "how available" always comes bundled with "how consistent, and measured against what promise."

Copies तभी मदद करती हैं जब वे स्वतंत्र रूप से fail होती हैं। और हर copy एक नई जगह है जहाँ data असहमत हो सकता है — इसलिए "कितना available" हमेशा "कितना consistent, किस promise के मुकाबले" के साथ बंधा होता है।

3. Key Points to Remember ☆

- ★ **Definition:** Availability = uptime = operational time ÷ time the system is supposed to be operational.
- ★ **The seven strategies group into three buckets:** add copies, spread the work, catch it early.
- ★ **Every "add copies" strategy forces a consistency decision.** Redundancy is not free, its price is paid in consistency.
- ★ **Redundancy without failure-domain diversity is fake redundancy.**
- ★ **The CAP theorem is the formal version of "consistency costs availability."**
- ★ **Availability is always measured against a specific promise on a specific component.**
- ★ **SLI / SLO / SLA are different things.**
- ★ **"Nines" of availability translate to concrete downtime budgets per year.**
- ★ **Consistency is a per-feature decision, not a global setting.**
- ★ **Commonly forgotten:** maintenance and monitoring are availability strategies too, not just architecture.

हिंदी अनुवाद – मुख्य बिंदु

परिभाषा: Availability = uptime = operational समय ÷ वह समय जितना सिस्टम को operational होना चाहिए।

सात रणनीतियाँ तीन समूहों में बँटती हैं: copies जोड़ना, काम बाँटना, जल्दी पकड़ना।

हर 'copies जोड़ना' रणनीति एक consistency फैसला मजबूर करती है। Redundancy मुफ्त नहीं, इसकी कीमत consistency में चुकानी पड़ती है।

Failure-domain diversity के बिना redundancy नकली redundancy है।

CAP theorem, 'consistency, availability की कीमत लेती है' का औपचारिक रूप है।

Availability हमेशा किसी खास component पर किए गए किसी खास promise के मुकाबले मापी जाती है।

SLI / SLO / SLA अलग-अलग चीज़ें हैं।

Availability की 'nines', हर साल के ठोस downtime budgets में बदल जाती हैं।

Consistency एक per-feature फैसला है, कोई global setting नहीं।

अक्सर भूली जाने वाली बात: maintenance और monitoring भी availability रणनीतियाँ हैं, सिर्फ architecture नहीं।

4. Real-World Examples

These illustrate well-known, publicly documented patterns — not confirmed claims about any company's current non-public internals.

ये well-known, publicly documented patterns को दर्शाते हैं — किसी company के मौजूदा non-public internals के बारे में पुष्ट दावे नहीं।

Databases with tunable consistency

हिंदी

Tunable consistency वाले Databases

ENGLISH

Cassandra and **DynamoDB** let you choose a consistency level per read or write — this makes the strong/weak/eventual choice a literal, per-query dial rather than an abstract idea. **MongoDB** replica sets have a similar knob through read preference and write concern settings.

हिंदी

Cassandra और **DynamoDB** आपको हर read या write के लिए एक consistency level चुनने देते हैं — इससे strong/weak/eventual का चुनाव एक अमूर्त विचार न रहकर, हर query पर एक असली dial बन जाता है। **MongoDB** के replica sets में read preference और write concern settings के ज़रिए एक समान विकल्प होता है।

Cloud infrastructure

हिंदी

क्लाउड इन्फ्रास्ट्रक्चर

ENGLISH

Cloud object stores commonly quote very high *durability* figures that are a different metric from *availability* — a classic interview distinction. Multi-AZ and multi-region patterns on **AWS**, **GCP**, and **Azure** directly implement "geographic distribution," and managed health checks implement "health monitoring."

हिंदी

Cloud object stores आमतौर पर बहुत ऊँचे *durability* आँकड़े बताते हैं, जो *availability* से एक अलग metric है — यह एक classic interview फ़र्क है। **AWS**, **GCP**, और **Azure** पर Multi-AZ और multi-region patterns, 'geographic distribution' का सीधा implementation हैं, और managed health checks, 'health monitoring' का implementation हैं।

Products and companies

हिंदी

प्रोडक्ट्स और कंपनियाँ

ENGLISH

Netflix is widely known for deliberately injecting failures into its own production systems (chaos engineering) to verify redundancy assumptions still hold. Messaging platforms like **WhatsApp** favor eventual consistency for message delivery. Ride-hailing and delivery platforms — **Uber**, **Careem**, **Talabat**, **Zomato** — tolerate a few seconds of staleness on driver-location updates while treating payment finalization more strictly. Payment platforms — **Razorpay**, **PhonePe**, **PayTM**, **Tabby**, **Tamara** — prefer strong consistency on ledger operations even at some availability cost.

हिंदी

Netflix अपने ही production systems में जान-बूझकर failures डालने (chaos engineering) के लिए जाना जाता है, ताकि redundancy की धारणाएँ जाँची जा सकें। **WhatsApp** जैसे messaging platforms, message delivery के लिए eventual consistency को प्राथमिकता देते हैं। Ride-hailing और delivery platforms — **Uber**, **Careem**, **Talabat**, **Zomato** — driver-location updates में कुछ सेकंड की staleness सह लेते हैं जबकि payment finalization को ज़्यादा सख्ती से संभालते हैं। Payment platforms — **Razorpay**, **PhonePe**, **PayTM**, **Tabby**, **Tamara** — कुछ availability कीमत पर भी ledger operations पर strong consistency को प्राथमिकता देते हैं।

 Caution / सावधानी

ENGLISH

Treat the company examples above as illustrations of well-known industry patterns, not verified statements about current non-public architecture.

हिंदी

ऊपर दिए गए company उदाहरणों को well-known industry patterns की मिसाल मानें, न कि मौजूदा non-public architecture के बारे में पुष्ट बयान।

5. When to Use / When NOT to Use

When to Invest Heavily in Availability

- User-facing, revenue- or safety-critical systems.
- A global user base with no safe maintenance window.
- Read-heavy workloads tolerant of staleness.

हिंदी:

- जब सिस्टम user-facing, revenue या safety के लिहाज़ से महत्वपूर्ण हो।
- जब users दुनिया भर में फैले हों, कोई सुरक्षित maintenance window न हो।
- जब read-heavy workloads staleness सह सकते हों।

When Extreme Availability Is Overkill

- Internal tools, batch pipelines, admin dashboards.
- Early-stage or low-traffic products.
- Systems where correctness must trump uptime.

हिंदी:

- Internal tools, batch pipelines, admin dashboards।
- शुरुआती चरण या कम-traffic वाले products।
- जहाँ correctness को uptime से ऊपर होना चाहिए।

When to Lean Toward Strong Consistency

- Financial transactions and account balances.
- Inventory/seat allocation where overselling causes harm.
- Data where "wrong but available" is worse than "briefly unavailable."

हिंदी:

- Financial transactions और account balances।
- Inventory/seat allocation जहाँ oversell नुकसान करता है।
- जहाँ 'गलत लेकिन available' 'थोड़ी देर unavailable' से बदतर है।

When to Lean Toward Eventual / Weak Consistency

- Social feeds, counters, recommendations.
- Search freshness, caching, analytics.
- Anywhere a few seconds of staleness causes no harm.

हिंदी:

- Social feeds, counters, recommendations।
- Search freshness, caching, analytics।
- जहाँ कुछ सेकंड की staleness से कोई नुकसान नहीं।

It depends on the requirements / यह requirements पर निर्भर करता है

ENGLISH

There is no universal answer. Availability targets and consistency levels should be set per component, based on business and user impact.

हिंदी

कोई universal जवाब नहीं है। Availability targets और consistency levels को per-component सेट किया जाना चाहिए, business और user पर असर के आधार पर।

6. Trade-offs

Option	Advantages	Disadvantages	Best Use Case
Strong Consistency	Every reader sees correct data	Higher latency; availability suffers	Payments, balances, allocation
Weak Consistency	Best availability/performance	Stale/conflicting data possible	Caches, ephemeral counters
Eventual Consistency	Good availability, converges	Brief stale-read windows	Feeds, catalogs, location updates

हिंदी अनुवाद — तालिका

विकल्प	फ़ायदे	नुकसान	सबसे उपयुक्त स्थिति
Strong Consistency	हर reader को सही data दिखता है	ज़्यादा latency; availability को नुकसान	Payments, balances, allocation
Weak Consistency	सबसे अच्छी availability/performance	Stale/conflicting data संभव	Caches, ephemeral counters
Eventual Consistency	अच्छी availability, converge होता है	Stale-read की छोटी अवधि	Feeds, catalogs, location updates

Dimension by dimension

ENGLISH

Performance: strong consistency adds coordination latency; weak/eventual avoids that wait, at the cost of temporary staleness.

हिंदी

Performance: Strong consistency coordination latency जोड़ती है; weak/eventual उस इंतज़ार से बचती है, अस्थायी staleness की कीमत पर।

ENGLISH

Cost: redundancy and geographic distribution multiply cost with the number of copies; three-to-five nines is disproportionately expensive.

हिंदी

Cost: Redundancy और geographic distribution, copies की संख्या के अनुपात में cost बढ़ाते हैं; तीन से पाँच nines तक जाना असमान रूप से महँगा है।

ENGLISH

Complexity & Operational Overhead: multi-region active-active needs failover automation, health checks, and runbooks — ongoing work.

हिंदी

Complexity और Operational Overhead: Multi-region active-active को failover automation, health checks, और runbooks चाहिए — लगातार चलने वाला काम।

ENGLISH

Development Effort: handling eventual consistency correctly (idempotency, conflict resolution) is real work, easy to underestimate.

हिंदी

Development Effort: Eventual consistency को सही तरीके से संभालना असली काम है, जिसे कम आँकना आसान है।

ENGLISH

User Experience: slow-but-correct and fast-but-briefly-wrong fail differently — the right choice depends on what the data is for.

हिंदी

User Experience: धीमा-लेकिन-सही और तेज़-लेकिन-थोड़ी देर के लिए-गलत, अलग-अलग तरीके से असफल होते हैं — सही चुनाव data के मक़सद पर निर्भर करता है।

7. Common Misconceptions

"High availability means the system never goes down."

"High availability का मतलब है सिस्टम कभी down नहीं होता।"

WHY IT'S WRONG

Availability is a statistical measure against a promise, not zero-downtime.

Availability एक promise के मुकाबले सांख्यिकीय माप है, शून्य-downtime नहीं।

CORRECT UNDERSTANDING

The goal is downtime within an agreed budget, kept short and contained.

मक़सद है सहमत बजट के भीतर downtime रखना, छोटा और सीमित।

INTERVIEW NUANCE

Talk about error budgets — downtime that can be deliberately "spent."

Error budgets की बात करें — downtime जिसे जान-बूझकर 'खर्च' किया जा सकता है।

"Adding redundant servers automatically gives you high availability."

"Redundant servers जोड़ देने से अपने आप high availability मिल जाती है।"

WHY IT'S WRONG

Shared failure domains can take all copies down together.

Shared failure domains सभी copies को एक साथ गिरा सकते हैं।

CORRECT UNDERSTANDING

Real redundancy needs failure-domain diversity.

असली redundancy के लिए failure-domain diversity चाहिए।

INTERVIEW NUANCE

Name the specific failure domain your redundancy protects against.

बताएं कि आपकी redundancy किस खास failure domain से बचाती है।

"You can have full availability and full consistency at the same time if you engineer hard enough."

"अगर पर्याप्त मेहनत से engineer किया जाए, तो पूरी availability और पूरी consistency एक साथ पाई जा सकती है।"

WHY IT'S WRONG

CAP theorem: during a partition, you must choose one.

CAP theorem: partition के दौरान, एक चुनना ही पड़ता है।

CORRECT UNDERSTANDING

The choice is per-feature, but can't be avoided entirely.

यह चुनाव per-feature है, लेकिन पूरी तरह टाला नहीं जा सकता।

INTERVIEW NUANCE

Naming CAP and the feature's side reads as senior.

CAP का नाम लेना और feature का पक्ष बताना senior लगता है।

""Eventual consistency" means the data is often wrong."

""Eventual consistency" का मतलब है data अक्सर गलत होता है।"

WHY IT'S WRONG

Convergence windows are typically milliseconds to seconds.

Convergence windows आमतौर पर milliseconds से सेकंड तक होती हैं।

CORRECT UNDERSTANDING

"Eventual" describes timing, not overall reliability.

'Eventual' समय को describe करता है, कुल reliability को नहीं।

INTERVIEW NUANCE

Describe how "read-your-own-write" is handled.

बताएं कि 'read-your-own-write' कैसे संभाला जाता है।

"A higher availability number is always a better engineering goal."

"ज़्यादा ऊँचा availability नंबर हमेशा एक बेहतर engineering लक्ष्य है।"

WHY IT'S WRONG

Each additional nine costs disproportionately more.

हर अतिरिक्त nine असमान रूप से ज़्यादा खर्च करती है।

CORRECT UNDERSTANDING

The target is set by business impact, not bigger-is-better.

Target business impact से तय होता है, बड़े नंबर की पसंद से नहीं।

INTERVIEW NUANCE

Ask "available for whom, measuring what."

पूछें 'किसके लिए available, क्या माप रहे हैं'।

8. Interview Questions

L1 Beginner

COMMON INTERVIEW QUESTION

What is availability in system design, and how is it typically measured?

System design में availability क्या है, और इसे कैसे मापा जाता है?

COMMON INTERVIEW QUESTION

What does "five nines" availability mean, and how much downtime does it allow per year?

"Five nines" का क्या मतलब है, और यह साल में कितना downtime allow करता है?

COMMON INTERVIEW QUESTION

What is a single point of failure, and how do you remove one?

Single point of failure क्या है, और आप इसे कैसे हटाते हैं?

FREQUENTLY REPORTED QUESTION

What is the difference between availability and reliability?

Availability और reliability में क्या अंतर है?

L2 Intermediate

FREQUENTLY REPORTED QUESTION · Amazon, Flipkart, Careem, Noon

How does load balancing improve availability?

Load balancing availability को कैसे बेहतर बनाता है?

COMMON INTERVIEW QUESTION

What is the difference between redundancy and replication?

Redundancy और replication में क्या अंतर है?

INTERVIEW-STYLE QUESTION

How would you design health checks for a fleet of servers?

आप servers के fleet के लिए health checks कैसे design करेंगे?

INTERVIEW-STYLE QUESTION

Why might two "redundant" servers still go down at the same time?

दो 'redundant' servers फिर भी एक साथ down क्यों हो सकते हैं?

L3 Senior

SENIOR-LEVEL FOLLOW-UP · Razorpay, PhonePe, PayTM, Tabby, Tamara

How does the CAP theorem relate to the availability-consistency trade-off?

CAP theorem, availability-consistency trade-off से कैसे जुड़ा है?

SENIOR-LEVEL FOLLOW-UP

How would you design a multi-region active-active architecture?

आप एक multi-region active-active architecture कैसे design करेंगे?

SENIOR-LEVEL FOLLOW-UP

When would you deliberately make part of a system less available?

आप जान-बूझकर सिस्टम के किसी हिस्से को कम available कब बनाएँगे?

SENIOR-LEVEL FOLLOW-UP

How do you decide which consistency level to use for a given piece of data?

आप किसी data के लिए कौन सा consistency level चुनें, यह कैसे तय करते हैं?

L4 Scenario-Based

SCENARIO-BASED QUESTION

Your SLA promises 99.95% but you keep missing it in a specific window. How do you investigate?

आपका SLA 99.95% का वादा करता है लेकिन आप एक खास window में चूक जाते हैं। आप कैसे जाँच करेंगे?

SCENARIO-BASED QUESTION

One availability zone goes completely offline. Walk through what happens.

एक availability zone पूरी तरह offline हो जाता है। बताइए क्या होता है।

SCENARIO-BASED QUESTION

Users see stale "in stock" data across regions. How do you explain and fix this?

Users को regions में stale "in stock" data दिखता है। आप इसे कैसे समझाएंगे और ठीक करेंगे?

SCENARIO-BASED QUESTION

How do you run a schema migration without downtime?

आप बिना downtime के schema migration कैसे चलाते हैं?

SCENARIO-BASED QUESTION · Flipkart, Noon, Talabat/HungerStation sale events

The data center serving a region goes down mid flash-sale. What's your failover plan?

Flash sale के बीच में एक region का data center down हो जाता है। आपका failover plan क्या है?

9. Interview Answers ☆

Q: What does "five nines" mean, and how much downtime does it allow?

Q (हिंदी): "Five nines" का क्या मतलब है, और यह कितना downtime allow करता है?

MODEL ANSWER

Five nines is 99.999% availability, roughly 5 minutes 15 seconds of downtime a year. I'd immediately ask what's being measured — the target should be set per component by business impact.

Five nines का मतलब है 99.999% availability, साल में लगभग 5 मिनट 15 सेकंड downtime। मैं तुरंत पूछूंगा कि क्या मापा जा रहा है — target हर component के लिए business impact से सेट होना चाहिए।

Q: How does load balancing improve availability?

Q (हिंदी): Load balancing availability को कैसे बेहतर बनाता है?

MODEL ANSWER

Health checks stop traffic to failing instances, so a single failure doesn't become a visible outage. It also lets me remove instances for maintenance without downtime.

Health checks fail हो रहे instances को traffic भेजना बंद कर देते हैं, इसलिए एक failure दिखने वाला outage नहीं बनती। यह मुझे बिना downtime के maintenance के लिए instances हटाने भी देता है।

Q: How does the CAP theorem relate to this lesson's trade-off?

Q (हिंदी): CAP theorem इस lesson के trade-off से कैसे जुड़ा है?

MODEL ANSWER

CAP is the formal statement: once you have replicas, a partition forces a choice between availability and consistency. I'd apply that per feature — strong for a ledger, eventual for a catalog.

CAP उसी चीज़ का औपचारिक बयान है: replicas होते ही, एक partition availability और consistency के बीच चुनाव मजबूर करता है। मैं यह per-feature लागू करूंगा — ledger के लिए strong, catalog के लिए eventual।

Requirement: payment balance must never be shown incorrectly / payment balance कभी गलत नहीं दिखना चाहिए

Decision: strong consistency for balance/ledger / balance/ledger के लिए strong consistency

Reason: wrong balance risks double-spend/overdraft / गलत balance से double-spend का खतरा

Trade-off: higher latency, reduced availability during partitions / ज़्यादा latency, कम availability

Alternative: eventual consistency with reconciliation — rejected due to risk / reconciliation के साथ eventual — जोखिम के कारण अस्वीकार

Q: Why might two "redundant" servers still fail together?

Q (हिंदी): दो 'redundant' servers फिर भी एक साथ क्यों fail हो सकते हैं?

MODEL ANSWER

If they share a failure domain, a single event takes both down. I'd place them in different zones/regions, and periodically test failover.

अगर वे एक failure domain share करते हैं, तो एक ही घटना दोनों को गिरा देती है। मैं उन्हें अलग-अलग zones/regions में रखूँगा, और समय-समय पर failover test करूँगा।

Q: One availability zone goes offline. What happens?

Q (हिंदी): एक availability zone offline हो जाता है। क्या होता है?

MODEL ANSWER

Health checks detect it in seconds; traffic reroutes; a replica in a healthy zone may be promoted. I'd then check replication lag and confirm no writes were lost.

Health checks सेकंडों में इसे detect कर लेते हैं; traffic फिर से भेज दिया जाता है; किसी healthy zone में एक replica promote हो सकता है। फिर मैं replication lag जाँचूँगा और पक्का करूँगा कि कोई write खोया तो नहीं।

Q: How do you run a schema migration without downtime?

Q (हिंदी): आप बिना downtime के schema migration कैसे चलाते हैं?

MODEL ANSWER

Expand-and-contract: add the new structure, write to both, backfill, switch reads, then remove the old — each step independently deployable.

Expand-and-contract: नया structure जोड़ें, दोनों में लिखें, backfill करें, reads शिफ्ट करें, फिर पुराना हटाएं — हर step अलग से deploy किया जा सकता है।

10. Follow-up Questions 🔥

Chain 1 — Choosing redundancy

Chain 1 — Redundancy चुनना

INTERVIEWER

"Why did you add redundant instances?"

"आपने redundant instances क्यों जोड़े?"

INTERVIEWER

"How do you know the redundant copy will actually take over?"

"आपको कैसे पता कि redundant copy काम संभाल लेगी?"

INTERVIEWER

"What if both copies are in the same data center?"

"अगर दोनों copies एक ही data center में हों तो?"

INTERVIEWER

"How do you test that failover actually works?"

"आप कैसे test करते हैं कि failover काम करता है?"

INTERVIEWER

"What's your recovery time when the primary fails?"

"Primary fail होने पर आपका recovery time क्या है?"

Chain 2 — Choosing a consistency level

Chain 2 — एक consistency level चुनना

INTERVIEWER

"Why did you choose eventual consistency here?"

"आपने यहाँ eventual consistency क्यों चुनी?"

INTERVIEWER

"What happens if a user reads stale data right after writing?"

"अगर user लिखने के तुरंत बाद stale data पढ़े तो क्या होता है?"

INTERVIEWER

"How long is your convergence window?"

"आपकी convergence window कितनी लंबी है?"

INTERVIEWER

"Which parts of the system can't tolerate that staleness?"

"सिस्टम के कौन से हिस्से वह staleness नहीं सह सकते?"

INTERVIEWER

"How would you fix read-your-own-writes for those parts?"

"आप उन हिस्सों के लिए read-your-own-writes कैसे ठीक करेंगे?"

Chain 3 — Measuring availability

Chain 3 — Availability मापना

INTERVIEWER

"How exactly are you calculating your 99.95%?"

"आप अपना 99.95% ठीक-ठीक कैसे calculate कर रहे हैं?"

INTERVIEWER

"Does that number count planned maintenance?"

"क्या उस नंबर में planned maintenance गिना जाता है?"

INTERVIEWER

"What if only one endpoint is down but the rest is fine?"

"अगर सिर्फ एक endpoint down हो लेकिन बाकी ठीक हो तो?"

INTERVIEWER

"How do you know your health checks aren't lying to you?"

"आपको कैसे पता कि health checks झूठ नहीं बोल रहे?"

11. Practical Design Exercise

Exercise 1 — Highly Available Redirect Service

Design a URL-shortener redirect service with a 99.99% availability target.

अभ्यास 1: एक URL-shortener redirect service design कीजिए जिसका 99.99% availability target हो।

Identify requirements, traffic, storage, architecture, bottlenecks, availability strategy, failure scenarios, and trade-offs first.

पहले requirements, traffic, storage, architecture, bottlenecks, availability strategy, failure scenarios, और trade-offs पहचानें।

Expected Thinking Process / अपेक्षित सोच-प्रक्रिया

- Translate 99.99% into a downtime budget (~52 min/year).
- Separate the critical redirect path from link-creation.
- Plan failure-domain diversity across zones.
- Redirects tolerate cache staleness — favor availability.
- Track the real SLI, not proxy metrics.

- 99.99% को एक downtime budget में बदलें (~52 मिनट/साल)।
- Critical redirect path को link-creation से अलग करें।
- Zones में failure-domain diversity की योजना बनाएं।
- Redirects cache staleness सह सकते हैं — availability को प्राथमिकता।
- असली SLI track करें, proxy metrics नहीं।

Reference Solution / संदर्भ समाधान

Stateless redirect servers across zones behind a load balancer with health checks. A cache holds hot mappings with eventual consistency. The database has a replica for failover; link creation tolerates stronger consistency.

Zones में stateless redirect servers, health checks वाले load balancer के पीछे। एक cache hot mappings को eventual consistency के साथ रखता है। Database में failover के लिए एक replica है; link creation ज़्यादा consistency सह सकता है।

Exercise 2 — Checkout That Must Stay Up but Never Oversell

An e-commerce checkout system must stay available during a flash sale, but must never oversell inventory.

अभ्यास 2: एक e-commerce checkout को flash sale में available रहना है, पर कभी oversell नहीं करना।

Identify requirements, traffic, storage, architecture, bottlenecks, availability/consistency strategy, failure scenarios, and trade-offs first.

पहले requirements, traffic, storage, architecture, bottlenecks, availability/consistency strategy, failure scenarios, और trade-offs पहचानें।

Expected Thinking Process / अपेक्षित सोच-प्रक्रिया

- Two consistency needs: browsing vs. stock-count.
- Separate "must never be wrong" from "fine to be stale."
- Consider a reservation pattern over a global lock.
- Keep browsing available and fast, independent of checkout.

- Consider the stock service becoming a bottleneck itself.

- दो consistency ज़रूरतें: browsing बनाम stock-count।
- 'कभी गलत नहीं' को 'stale होना ठीक है' से अलग करें।
- Global lock की जगह reservation pattern पर विचार करें।
- Browsing को checkout से स्वतंत्र, available और तेज़ रखें।
- सोचें कि stock service खुद bottleneck बन सकती है।

Reference Solution / संदर्भ समाधान

Browsing runs on cached, eventually-consistent read replicas. Stock-count uses a single authoritative service with strong consistency and a short-lived reservation pattern, keeping the strict consistency requirement on the smallest critical path.

Browsing, cached, eventually-consistent read replicas पर चलती है। Stock-count एक अकेली authoritative service से strong consistency और एक अल्पकालिक reservation pattern के साथ संभाली जाती है, जिससे सख्त consistency requirement सबसे छोटे critical path पर केंद्रित रहती है।

12. Quick Interview Revision Sheet

30-Second Revision

Availability = uptime against a promise. Seven strategies group into add copies, spread the work, catch it early. Every strategy adds copies, forcing a consistency choice.

Availability = किसी promise के मुकाबले uptime। सात रणनीतियाँ copies जोड़ना, काम बाँटना, जल्दी पकड़ना में बँटती हैं। हर रणनीति copies जोड़ती है, जो एक consistency चुनाव मजबूर करती है।

2-Minute Revision

- Availability = operational time ÷ time it should be operational, against a named promise.
- Seven strategies: add copies, spread the work, catch it early.
- Redundancy needs failure-domain diversity.
- CAP theorem: choose availability or consistency during a partition.
- Strong: identical, costs availability. Weak: fastest. Eventual: converges.
- SLI/SLO/SLA are different things.
- Higher targets cost disproportionately more — set per component.

- Availability = operational समय ÷ जितना operational होना चाहिए, एक नामित promise के मुकाबले।
- सात रणनीतियाँ: copies जोड़ना, काम बाँटना, जल्दी पकड़ना।
- Redundancy को failure-domain diversity चाहिए।
- CAP theorem: partition के दौरान availability या consistency चुनें।
- Strong: एक जैसा, availability की कीमत। Weak: सबसे तेज़। Eventual: converge होता है।
- SLI/SLO/SLA अलग-अलग चीज़ें हैं।
- ऊँचे targets असमान रूप से ज़्यादा खर्च करते हैं — per component सेट करें।

Interview Cheat Sheet – The Nines

Availability	Nines	Downtime / Year
99%	Two nines	~3.65 days
99.9%	Three nines	~8.76 hours
99.99%	Four nines	~52.6 minutes
99.999%	Five nines	~5.3 minutes
99.9999%	Six nines	~31.6 seconds

हिंदी अनुवाद

Availability	Nines	सालाना Downtime
99%	दो nines	~3.65 दिन
99.9%	तीन nines	~8.76 घंटे
99.99%	चार nines	~52.6 मिनट
99.999%	पाँच nines	~5.3 मिनट
99.9999%	छह nines	~31.6 सेकंड

13. Senior Engineer Perspective

How a Senior Engineer Should Think About This

ENGLISH

A senior engineer treats "what's our availability target" as an incomplete question until it's scoped: target for which component, measured how, against what promise. They resist the reflex to chase more nines everywhere, because each additional nine costs disproportionately more — and they can usually estimate that cost concretely enough to make the trade-off a real business conversation.

हिंदी

एक senior engineer 'हमारा availability target क्या है' को तब तक अधूरा सवाल मानता है जब तक उसका scope तय न हो: किस component के लिए, कैसे मापा गया, किस promise के मुकाबले। वे हर जगह ज़्यादा nines के पीछे भागने की सहज प्रवृत्ति का विरोध करते हैं, क्योंकि हर अतिरिक्त nine असमान रूप से ज़्यादा खर्च करती है — और वे उस cost का इतना ठोस अनुमान लगा सकते हैं कि trade-off एक असली business बातचीत बन जाए।

ENGLISH

They treat redundancy as something to verify, not assume — periodically testing failover rather than trusting a diagram. They think in failure domains: which specific event does this redundancy protect against, and which one would still take the whole system down.

हिंदी

वे redundancy को verify करने वाली चीज़ मानते हैं, मान लेने वाली नहीं — diagram पर भरोसा करने के बजाय समय-समय पर failover test करते हैं। वे failure domains में सोचते हैं: यह redundancy किस खास घटना से बचाती है, और कौन सी घटना फिर भी पूरे सिस्टम को गिरा देगी।

ENGLISH

They treat consistency as a per-feature decision, not a global setting — and can articulate which side of the CAP trade-off each critical piece of data sits on. They use error budgets rather than treating every minute of downtime as equally bad.

हिंदी

वे consistency को एक per-feature फैसला मानते हैं, कोई global setting नहीं — और बता सकते हैं कि हर महत्वपूर्ण data CAP trade-off के किस तरफ बैठा है। वे हर मिनट के downtime को बराबर बुरा मानने के बजाय error budgets इस्तेमाल करते हैं।

ENGLISH

Observability is planned around the actual SLI users care about, not proxy signals. And maintainability is judged by whether failover procedures are documented and rehearsed well enough that any on-call engineer can execute them at 3 a.m.

हिंदी

Observability असली SLI के इर्द-गिर्द बनाई जाती है, proxy signals के नहीं। और maintainability को इस आधार पर आँका जाता है कि क्या failover प्रक्रियाएँ इतनी अच्छी तरह documented और rehearsed हैं कि कोई भी on-call engineer उन्हें रात 3 बजे भी अंजाम दे सके।

The senior-level default / Senior-level डिफ़ॉल्ट

ENGLISH

Never present availability as "just add redundancy" or consistency as "always strong is safest." The correct senior answer is always: it depends on the requirements — Requirement → Decision → Reason → Trade-off → Alternative.

हिंदी

Availability को कभी 'बस redundancy जोड़ दो' के रूप में पेश न करें। सही senior जवाब हमेशा यही है: यह requirements पर निर्भर करता है — Requirement → Decision → Reason → Trade-off → Alternative।

General + Technical Words Glossary

Advanced words and phrases used across this Interview Notes document.

General English Words

English Word / Term	Hindi Meaning	Simple Explanation
scoped	सीमांकित / परिभाषित दायरा	Defined with clear boundaries about what's included.
reflex	सहज प्रवृत्ति	An automatic, unthought-out reaction or habit.
disproportionately	असमान रूप से	At an unfair or unequal rate compared to something else.
verify	सत्यापित करना	To check that something is actually true.
rehearsed	अभ्यास किया हुआ	Practiced in advance so it can be done smoothly.
bounded	सीमाबद्ध	Kept within known limits.
converge	एकत्रित होना	Gradually coming together to the same point.
contention	प्रतिस्पर्धा / टकराव	Multiple processes competing for the same resource.
authoritative	आधिकारिक / प्रामाणिक	The single trusted source of truth for some data.
degrade (gracefully)	धीरे-धीरे क्षीण होना	Reducing functionality in a controlled, acceptable way.
blip	छोटी रुकावट	A brief, minor disruption.
backfill	पुराने data को भरना	To fill in historical/missing data after a change.
cutover	बदलाव का क्षण	The moment of switching fully from old to new.
tolerant (of staleness)	सहनशील	Able to accept slightly outdated data without harm.
calibrate	अंशांकन करना	To adjust something to be accurate or appropriate.
correlated failure	सहसंबंधित विफलता	Multiple failures happening together from a shared cause.
game day	गेम डे	A planned exercise simulating failures to test readiness.

Technical Words

English Word / Term	Hindi Meaning	Simple Explanation
CAP Theorem	CAP प्रमेय	A distributed system can't fully guarantee Consistency, Availability, and Partition tolerance together.
SLI (Service Level Indicator)	SLI	The actual metric measured, e.g. successful request rate.
SLO (Service Level Objective)	SLO	An internal target for an SLI.
SLA (Service Level Agreement)	SLA	An external, often contractual, promise about performance.
Error Budget	एरर बजट	The allowed amount of downtime implied by an availability target.
Failure Domain	फेलियर डोमेन	A boundary within which a single failure can spread.
Multi-AZ	मल्टी-AZ	Deployment spread across multiple availability zones.
Active-Active	एक्टिव-एक्टिव	Multiple regions/instances all serving live traffic together.
Failover	फेलओवर	Automatically switching to a backup when the primary fails.
Quorum	कोरम	Minimum number of nodes that must agree before success.
Read-Your-Own-Writes	रीड-योर-ओन-राइट्स	A guarantee that a user always sees their own recent write.
Reservation Pattern	रिज़र्वेशन पैटर्न	Temporarily holding a resource during a transaction.
Chaos Engineering	केऑस इंजीनियरिंग	Deliberately introducing failures to test resilience.
Durability	ड्यूरैबिलिटी	Guarantee that stored data won't be lost, distinct from availability.
Expand-and-Contract	एक्सपैंड-एंड-कॉन्ट्रैक्ट	A safe migration pattern: add, migrate, then remove old.
Replica Promotion	रेप्लिका प्रमोशन	Making a standby replica the new primary after failure.
Replication Lag	रेप्लिकेशन लैग	Delay between a write on primary and it appearing on replica.

English Word / Term	Hindi Meaning	Simple Explanation
Health Check	हेल्थ चेक	An automated probe verifying an instance works correctly.